

CI1164 – Introdução à Computação Científica

Prof. Armando Delgado

Exercícios de Revisão para Prova 01

GABARITO

Questão 1

Considere um equipamento cujo sistema de ponto flutuante **normalizado** é SPF(2,4,-5,5), ou seja, de **base 2**, possui **4 dígitos na mantissa**, **menor expoente -5** e **maior expoente 5**. Os números abaixo são fornecidos a este sistema:

$$(a) \quad 0.1011 \times 2^4 \qquad (b) \quad 0.1101 \times 2^{-1} \qquad (c) \quad 0.1110 \times 2^1$$

Qual é o resultado das seguintes operações, considerando que a máquina efetua o truncamento dos resultados. Calcule também para cada item o valor exato (sem considerar truncamento) .

1) $(a+b)+c$

$$x = (1011 + 0,01101) + 1,110 = 1101,00101 = 13,15625_{10}$$

$$\bar{x} = (0,1011 * 2^4 + 0,000001101 * 2^4) + 0,1110 * 2^1 = 0,1011 * 2^4 + 0,0001110 * 2^4 = 0.1100 * 2^4 = 12_{10}$$

2) $a+(b+c)$

$$x = 1011 + (0,01101 + 1,110) = 1101,00101 = 13,15625_{10}$$

$$\begin{aligned} \bar{x} &= 0,1011 * 2^4 + (0,001101 * 2^1 + 0,1110 * 2^1) = \\ &0,1011 * 2^4 + 1,0001 * 2^1 = 0,1011 * 2^4 + 0,1000 * 2^2 = 0,1011 * 2^4 + 0,0010 * 2^4 = 0.1101 * 2^4 = 13_{10} \end{aligned}$$

3) Explique a diferença de resultados verificadas nos itens (1) e (2)

No item (1) a soma dos elementos **a** e **b** causa um cancelamento subtrativo pois a mantissa não é capaz de representar o valor de **b** quando seu expoente tem que ser igualado ao expoente de **a**. Este cancelamento tem efeito menor no item (2) porque ao menos parte do valor de **b** pode ser somado a **c** e não é perdido na subsequente operação com **a**.

Questão 2

O algoritmo abaixo pode ser utilizado para calcular com quantos dígitos um computador trabalha.

```
ε ← 1.0
j = 1
Enquanto (1.0 + ε > 1.0) faça
    ε ← ε / 2.0
    j ← j + 1

Escreva o valor de j
```

Explique por que o algoritmo não entra em laço infinito.

R: O valor de ε é dividido ao meio a cada iteração, o que equivale a reduzir o expoente em uma unidade a cada iteração (consequentemente um dígito da mantissa é retirado da precisão do número ε no momento da soma). O laço termina obrigatoriamente porque em algum momento todos os dígitos da mantissa de ε serão deslocados para igualar o expoente de 1.0, deixando $\varepsilon = 0.0$ na operação de soma.

Questão 3

Considere as duas expressões equivalentes abaixo para calcular a abscissa da interseção da reta que passa pelos pontos (x_0, y_0) e (x_1, y_1) com o eixo x :

$$(a) \quad x = \frac{x_0 y_1 - x_1 y_0}{y_1 - y_0} \qquad (b) \quad x = x_0 - \frac{(x_1 - x_0) y_0}{y_1 - y_0}$$

i. Usando os pontos $(0.131 \times 10^1, 0.324 \times 10^1)$ e $(0.193 \times 10^1, 0.476 \times 10^1)$ calcule o valor de x em um **SPF(10,3,-5,5)**. Calcule os erros absolutos e relativos destas aproximações.

(a)

$$x_a = \frac{1,31 * 4,76 - 1,93 * 3,24}{4,76 - 3,24} = \frac{6,2356 - 6,2532}{1,52} = \frac{-0,0176}{1,52} = -0,011578947$$

$$\begin{aligned} \bar{x}_a &= \frac{0.131 \times 10^1 * 0.476 \times 10^1 - 0.193 \times 10^1 * 0.324 \times 10^1}{0,476 \times 10^1 - 0,324 \times 10^1} = \frac{0,0623 \times 10^1 - 0,0625 \times 10^1}{0,152 \times 10^1} \\ &= \frac{0,623 \times 10^0 - 0,625 \times 10^0}{0,152 \times 10^1} = \frac{-0,002 \times 10^0}{0,152 \times 10^1} \\ &= -0,00131 \end{aligned}$$

$$EA = |x_a - \bar{x}_a| = |-0,011578947 - (-0,00131)| = 0,010268947$$

$$ER = \frac{EA}{|x_a|} \times 100 = \frac{0,010268947}{0,011578947} \times 100 = 88,6864\%$$

(b)

$$x_b = 1,31 - \frac{(1,93 - 1,31) * 3,24}{4,76 - 3,24} = 1,31 - \frac{0,62 * 3,24}{1,52} = 1,31 - \frac{2,0088}{1,52} = 1,31 - 1,321578947 \\ = -0,011578947$$

$$\bar{x}_b = 0,131 \times 10^1 - \frac{(0,193 \times 10^1 - 0,131 \times 10^1) * 0,324 \times 10^1}{0,476 \times 10^1 - 0,324 \times 10^1} = 0,131 \times 10^1 - \frac{0,062 \times 10^1 * 0,324 \times 10^1}{0,152 \times 10^1} \\ = 0,131 \times 10^1 - \frac{0,020 \times 10^1}{0,152 \times 10^1} = 0,131 \times 10^1 - 0,131 \times 10^1 \\ = 0,000$$

$$EA = |x_b - \bar{x}_b| = |-0,011578947 - 0,00| = 0,011578947$$

$$ER = \frac{EA}{|x_b|} \times 100 = \frac{0,011578947}{0,011578947} \times 100 = 100\%$$

ii. Qual dos dois métodos é melhor? Justifique.

O primeiro método é o melhor, pois o efeito do cancelamento subtrativo é menor do que no segundo método.

Questão 4

Considere os métodos numéricos estudados nesta disciplina e o cálculo de erros apresentados abaixo.

i) Dada uma função $f(x)$ definida e contínua no intervalo I , chamamos de zero (ou raiz) da função a todo $\alpha \in I \mid f(\alpha) = 0$. Considere $x_i \approx \alpha$ o resultado da i-ésima iteração de algum método numérico para o cálculo de α . Indique em que situações cada uma das formas de cálculo de erro abaixo é mais adequada:

(a) $|f(x_i)|$: Quando a função próxima da raiz faz com que o método não produza valores que convergem rápido. Este erro não nos diz nada com relação à raiz propriamente dita e tem pouca utilidade para funções mal comportadas na proximidade da raiz.

(b) $|x_b - x_a|$, onde $x_a \leq \alpha \leq x_b$: Quando o método lida com intervalos contendo a raiz e este intervalo vai se estreitando sempre (bissecção e falsa posição)

ii) Seja um Sistema de Equações Lineares da forma $Ax = b, A \in \mathbb{R}^{n \times n}, \{x, b\} \in \mathbb{R}^n$ e $\bar{x}^{(k)} \approx x$ o resultado da k-ésima iteração de algum método para a solução de sistemas lineares. Lembrando que o resíduo é definido $r = b - A\bar{x}^{(k)}$, indique em que situações cada uma das formas de cálculo de erro abaixo é mais adequada:

d) $\|x\|_\infty = \max(|\bar{x}_i^{(k)} - \bar{x}_i^{(k-1)}|), i = 1, 2, \dots, n$: Quando se deseja ter indicação da contribuição de

cada incógnita para o erro.

e) $\|r\|_2 = \sqrt{r_1^2 + r_2^2 + \dots + r_n^2}$: Quando se deseja saber se o S.L. está satisfeito de forma geral. O que não sabemos é quanto cada variável contribui para cada erro.

iii) Sejam x_d e $raiz$ a representação em ponto flutuante IEEE754 (double) dos valores de x_i e α do item (i), respectivamente. Considerando ainda que o método numérico esteja convergindo, explique porque o laço a seguir (em linguagem C) não é uma opção para testar a convergência do método.

```
while( fabs(xd - raiz) > 0.0 )  
{  
    ...  
}
```

Devido à representação IEEE754, a diferença $xd - raiz$ pode nunca chegar a ser 0.0, mesmo havendo convergência do método. Portanto o valor de **fabs()** nunca será igual a zero e o laço nunca termina.

Questão 5

Observe o trecho de código a seguir, considere as variáveis **soma1** e **soma2** e responda:

```
float soma1=0.0f, soma2=0.0f;  
  
for (int i=1; i<=200; ++i)  
    soma1 += 1.0f / (i*i);  
  
for (int i=200; i>=1; --i)  
    soma2 += 1.0f / (i*i);
```

a) Qual variável terá o valor mais exato? Por que isso ocorre?

soma2 porque adicionar pequenos valores a uma soma com acumulador implica em erros de arredondamento devido ao cancelamento subtrativo. Iniciando pelos menores valores diminui a ocorrência de erros porque o somador não é muito maior do que os valores sendo somados a ele.

b) A precisão das variáveis é a mesma? Justifique sua resposta.

Sim, pois ambas utilizam a mesma representação em ponto flutuante, ou seja, possuem a mesma quantidade de dígitos na mantissa. Entretanto, **soma1** terá menos dígitos significativos do que **soma2**.

Questão 6

Considere um equipamento cujo sistema de ponto flutuante (SPF) **normalizado** de **base 2** possui **4 dígitos na mantissa**, **menor expoente -5** e **maior expoente 5** (SPF(2,4,-5,5)). Para este sistema:

a) Qual a diferença entre o menor número positivo representável e o próximo número, imediatamente maior (em base 2)?

$\min = ,1000 \times 2^{-5}$, $\text{prox} = ,1001 \times 2^{-5}$, $\text{diff} = ,0001 \times 2^{-5} = 1 \times 2^{-9}$

- b) Qual a diferença entre o maior número positivo representável e o número anterior, imediatamente menor (em base 2)?

$$\text{max} = ,1111 \times 2^5, \text{ antes} = ,1110 \times 2^5, \text{ diff} = ,0001 \times 2^5 = 1 \times 2^1$$

- c) Qual é o maior número inteiro ímpar que este sistema pode representar (em base 2 ou base 10)?

$$\text{maxImpar} = 0,1111 \times 2^4$$

- d) Explique uma implicação de suas respostas anteriores.

(a) Existem infinitos números reais no intervalo entre o menor e o próximo positivo, entretanto eles não podem ser representados no SPF em questão devido ao número limitado de dígitos na mantissa. Desta forma, qualquer número neste intervalo será truncado/arredondado para o menor ou para o próximo valor.

(b) O mesmo acontece para valores entre o maior valor e o anterior.

(c) Em ponto flutuante, apenas números inteiros pares podem ser representados a partir do maior inteiro ímpar representável em PF.

Observa-se que sempre há 5 números em ponto flutuante entre duas potências de dois:

- Para cada aumento na potência de 2, a quantidade de inteiros representados duplica, mas a quantidade de números em ponto flutuante é constante.
- A precisão do número em ponto flutuante é proporcional à sua magnitude. Quanto maior o número, menor a precisão.
- À medida em que o valor aumenta, diminui a precisão do número em ponto flutuante.

Questão 7

Escreva uma função em linguagem C que receba como parâmetros de entrada as diagonais de uma matriz tri-diagonal, o vetor de termos independentes $\vec{b}$, o vetor com os valores iniciais para $\vec{x}$, a ordem n da matriz, e um valor de tolerância tol , e devolva sua solução $\vec{x}$ usando o método de Gauss-Seidel até 5 (cinco) iterações. Se existir alguma condição ou propriedade para os vetores das diagonais e termos independentes, isto deve ser indicado na resposta.

```
/* double dp[N],
   double ds[N], ds[N-1] = 0
   double di[N], di[0] = 0
   double Xi[N+1], Xi[0] = 0, X = Xi+1
*/
void gaussSeidel( double *ds, double *dp, double *di, double *b,
                  int n, double *X)
{
    int i, j;
    double s;
```

```

for (j=0; j < 5; ++j) {
    for (i=0; i < n; ++i) {
        s = di[i]*X[i-1] + ds[i]*X[i+1];
        X[i] = (b[i] - s)/dp[i];
    }
}
}

```

Questão 8

Seja um Sistema de Equações Lineares da forma $Ax=b, A \in \mathbb{R}^{n \times n}, \{x, b\} \in \mathbb{R}^n$. Para a solução deste sistema, quantas operações aritméticas são realizadas EM CADA ITERAÇÃO dos Métodos de Jacobi e de Gauss-Seidel quando:

O algoritmos Gauss-Seidel e Jacobi executam o mesmo número de operações aritméticas, o que muda é a origem dos valores das variáveis do SL (iteração atual x iteração anterior).

$$x_i^{(k)} = \frac{1}{a_{ii}} \left[b_i - \sum_{j=1, j \neq i}^n a_{ij} \cdot x_j^{(k-1)} \right], \quad i=1,2,\dots,n \quad (\text{Jacobi})$$

$$x_i^{(k)} = \frac{1}{a_{ii}} \left[b_i - \sum_{j=1, j \neq i}^{i-1} a_{ij} \cdot x_j^{(k)} - \sum_{j=i+1, j \neq i}^n a_{ij} \cdot x_j^{(k-1)} \right], \quad i=1,2,\dots,n \quad (\text{Gauss-Seidel})$$

a) **A** é uma matriz $n \times n$?

Para cada equação do SL são efetuados **(n-1)** produtos, **(n-1)** somas e **1** (uma) divisão, isto é, **2n-1** operações aritméticas (em ponto flutuante). Cada ITERAÇÃO DO MÉTODO (cálculo de todos os valores x_i), portanto, realiza **$n(2n - 1) = 2n^2 - n$** operações aritméticas. Para **n** muito grandes, isto é da ordem de **$2n^2$** operações.

b) **A** é uma matriz de banda k -diagonal?

Para uma matriz k -diagonal, para cada equação do SL são efetuados **(k-1)** produtos, **(k-1)** somas e **1** (uma) divisão, isto é, **2k-1** operações aritméticas (em ponto flutuante). Cada ITERAÇÃO DO MÉTODO (cálculo de todos os valores x_i), portanto, realiza **$n(2k - 1) = 2kn - n$** operações aritméticas. Para **n** muito grandes, isto é da ordem de **$2kn$** operações.

c) Em quais condições estes métodos iterativos serão competitivos com o Método da Eliminação Gaussiana (sem considerar pivotamento)?

Por competitivo entenda-se equivalentes, ou a partir de que condições um executa mais operações que outro.

Na eliminação de Gauss, a quantidade de operações é **$(2/3)n^3 + O(n^2)$** para matrizes **$n \times n$** , ou seja, da ordem de **$O(n^3)$** operações. Para sistemas k -diagonais o número de operações é da ordem de **$O(kn^2)$** .

Os métodos iterativos são competitivos (ou calculam menos operações aritméticas) sempre que o número de iterações for menor do que **n**. A partir do momento em que

são necessárias aproximadamente n iterações para convergência, eles deixam de ser competitivos.

Questão 9

a) Explique como a inversa de uma matriz $A, n \times n$, pode ser obtida através da resolução de n sistemas lineares $n \times n$.

R: Seja M a inversa da matriz A tal que $A.M = I$. Podemos calcular cada coluna k da matriz M resolvendo um sistema linear $A.m_k = I_k$ para $k=1,2,\dots,n$

b) Entre o método da Eliminação de Gauss e a decomposição LU, qual o mais indicado para este caso? Justifique.

R: A decomposição LU é mais indicada pois todos n sistemas lineares a serem resolvidos compartilham a mesma matriz de coeficientes A . Desta forma, ela precisa ser fatorada apenas uma vez.

Questão 10

Matrizes tridiagonais são aquelas em que apenas os elementos da diagonal principal, e os elementos das diagonais imediatamente acima e abaixo são não nulos. Sistemas lineares com matrizes de coeficientes tridiagonais são bastante comuns na solução de problemas de computação científica.

OPÇÃO A

a) Elabore uma estrutura de dados em linguagem C para armazenar um sistema linear com matriz de coeficientes tridiagonal, que seja eficiente para resolução pelo método de Gauss-Seidel;

```
struct t_SistLinear3Diag {
    double *dp, *ds, *di; /* Diagonal principal, superior */
    double *b; /* Termos independentes */
    unsigned int n; /* numero de equações do SL */
};
```

b) Implemente uma função em C que resolva pelo método de Gauss-Seidel um sistema linear tridiagonal;

```
#define MAXIT 100
```

```
void gaussSeidel (struct t_SistLinear3Diag *SL, double *x, double
erro)
{
    double norma, diff, xk;
    int k = 1, i;
    do {
        // primeira equação fora do laço
        i = 0;
        xk = (SL->b[i] - SL->ds[i]*x[i+1]) / SL->dp[i];
```

```

norma = fabs(xk - x[0]);
x[i] = xk;

// equações centrais
for (i=1; i<SL->n-1; ++i) {
    xk = (SL->b[i] - SL->ds[i]*x[i+1] - SL->di[i]*x[i-1]) / SL->dp[i];
    // Calcula norma || x(k) - x(k-1) ||
    diff = fabs(xk - x[i]);
    norma = (diff > norma) ? (diff) : (norma);
    x[i] = xk;
}

// ultima equação fora do laço
xk = (SL->b[i] - SL->di[i]*x[i-1]) / SL->dp[i];
diff = fabs(xk - x[i]);
norma = (diff > norma) ? (diff) : (norma);
x[i] = xk;

++k;
} while (norma > erro && k < MAXIT) ;
}

```

OPÇÃO B

- a) Elabore uma estrutura de dados em linguagem C para armazenar um sistema linear com matriz de coeficientes tridiagonal, que seja eficiente para resolução pelo método de Gauss-Seidel;

```

typedef struct {
    double *A[3];
    double *b;
    int n;
} SLTridiag_t;

```

- b) Implemente uma função em C que resolva pelo método de Gauss-Seidel um sistema linear tridiagonal;

```

void GaussSeidelTridiag ( SLTridiag sl, double *x, int maxIter ) {
    int i, iter, INF=0, PRI=1, SUP=2;

    for (iter=0; iter < maxIter; ++iter)
    {
        x[0] = (sl.b[0] - sl[SUP][0]*sl.x[1]) / sl.A[PRI][0];
        for (i=1; i < sl.n-1; ++i)
        {
            x[i] = (sl.b[i] - sl[INF][i]*sl.x[i-1] - sl[SUP][i]*sl.x[i+1])
                    / sl.A[PRI][i];
        }
        x[sl.n-1] = (sl.b[sl.n-1] - sl[INF][sl.n-1]*sl.x[sl.n-2])
                    / sl.A[PRI][sl.n-1];
    }
}

```


}

Questão 11

Defina o que é um sistema linear bem condicionado (estável) e o que é um sistema linear mal condicionado.

R: Num sistema bem condicionado pequenas alterações na matriz de coeficientes não alteram a convergência ou o resultado do método (a menos de erros numéricos). Num sistema mal condicionado ocorre o contrário.

Questão 12

Quando a decomposição LU é vantajosa computacionalmente se comparada ao Método da Eliminação de Gauss?

R: A decomposição LU é vantajosa sempre que for necessário resolver mais de um sistema linear com a mesma matriz de coeficientes, i.e., apenas alteram-se os valores dos termos independentes das equações.

Questão 13

Considere o sistema linear 3-diagonal $\mathbf{Ax} = \mathbf{B}$ abaixo, ao qual se aplica o Método de Gauss-Seidel, otimizado para este tipo de matriz:

$$\begin{bmatrix} 2 & 59 & 0 & 0 & 0 & 0 & 0 \\ 34 & 10 & 12 & 0 & 0 & 0 & 0 \\ 0 & 3 & 117 & 13 & 0 & 0 & 0 \\ 0 & 0 & 25 & 14 & 90 & 0 & 0 \\ 0 & 0 & 0 & 31 & 2 & 27 & 0 \\ 0 & 0 & 0 & 0 & 13 & 120 & 31 \\ 0 & 0 & 0 & 0 & 0 & 21 & 24 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{bmatrix} = \begin{bmatrix} 6 \\ 1 \\ 4 \\ 13 \\ 20 \\ 32 \\ 6 \end{bmatrix}$$

- (a) O que se pode afirmar sobre a convergência deste método quando aplicado na solução deste sistema linear? Explique sua resposta.
- (b) É possível aplicar operações de forma a se obter um sistema linear equivalente cuja solução seja possível aplicando-se este método? Explique sua resposta.

(a) A matriz $\mathbf{A}$ não é diagonalmente dominante, de forma que não se pode garantir que o método irá convergir.

(b) É possível com a troca de linhas, mas então o SL resultante deixa de ser 3-diagonal, impossibilitando o uso da otimização do método para matrizes 3-diagonais:

$$\begin{bmatrix} 34 & 10 & 12 & 0 & 0 & 0 & 0 \\ 2 & 59 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 117 & 13 & 0 & 0 & 0 \\ 0 & 0 & 0 & 31 & 2 & 27 & 0 \\ 0 & 0 & 25 & 14 & 90 & 0 & 0 \\ 0 & 0 & 0 & 0 & 13 & 120 & 31 \\ 0 & 0 & 0 & 0 & 0 & 21 & 24 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{bmatrix} = \begin{bmatrix} 1 \\ 6 \\ 4 \\ 20 \\ 13 \\ 32 \\ 6 \end{bmatrix}$$

Questão 14

Considere a tentativa abaixo para comparar dois valores em ponto flutuante usando **ULPs**. Identifique o que está incorreto no cálculo de ULPs e mostre como resolver o problema.

```
// devolve 1 se x e y estão a no máximo 1 ULP de distância
int quase_iguais(float x, float y)
{
    if (x == y)
        return 1;

    int xi = (int) x;
    int yi = (int) y;

    return (xi-1 <= yi && yi <= xi+1);
}
```

A conversão de **float** para **int** (via **cast**) muda a sua representação binária, de forma que **xi-1** e **xi+1** NÃO SÃO os **float**'s que, respectivamente, antecedem e sucedem o número **x**, e podem estar a muitos ULPs de distância. A verificação não pode alterar a representação binária dos números.

VERSÃO 1:

```
typedef union {
    int i;
    float f;
} Float_t;

int quase_iguais(float x, float y)
{
    if (x == y)
        return 1;

    Float_t X, Y;
    X.f = x;
    Y.f = y;

    return (X.i-1 <= Y.i && Y.i <= X.i+1);
}
```

```
}
```

VERSÃO 2:

```
int quase_iguais(float x, float y)
{
    if (x == y)
        return 1;

    int *xi = (int *) &x;
    int *yi = (int *) &y;

    return (*xi-1 <= *yi && *yi <= *xi+1);
}
```

Atenção: nas duas versões acima, não se garante a comparação correta (isto é, considerando 1 ULP) para números **denormalizados** próximos de zero. Nesses casos, pode haver **overflow** nas operações com inteiros e problemas de comparação quando o número denormalizado tem sinal diferente do zero (ex: menor número denormalizado positivo é -0).

Questão 15

Escreva as equações do sistema linear gerado quando a equação diferencial ordinária é $\frac{d^2 y}{dx^2} = 6x - 0.5x^2$ resolvida para um problema de valor de contorno $y(0)=0$, $y(12)=0$, $0 \leq x \leq 12$ e uma discretização com passo $h=4$ utilizando diferenças finitas centrais para aproximar a derivada $\left(\frac{dy}{dx} \Big|_i \simeq \frac{y_{i+1} - y_{i-1}}{2(h)} \right.$ e $\left. \frac{d^2 y}{dx^2} \Big|_i \simeq \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} \right)$.

OBS.: Demonstrar todos os passos até a obtenção das equações.

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} = 6x_i - 0,5x_i^2, \quad x_i = \{0, 4, 8, 12\}$$

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{(4)^2} = 6x_i - 0,5x_i^2 \Rightarrow y_{i-1} - 2y_i + y_{i+1} = 96x_i - 8x_i^2$$

$$\begin{aligned} y_0 - 2y_1 + y_2 &= 256, \quad i=1, x_1=4 & \Rightarrow & -2y_1 + y_2 = 256 - y_0 & \Rightarrow & -2y_1 + y_2 = 256 \\ y_1 - 2y_2 + y_3 &= 256, \quad i=2, x_2=8 & \Rightarrow & y_1 - 2y_2 = 256 - y_3 & \Rightarrow & y_1 - 2y_2 = 256 \end{aligned}$$

Questão 16

Escreva as equações do sistema linear gerado quando a equação diferencial ordinária

$$\frac{d^2 y}{dx^2} + x \cdot \frac{dy}{dx} - x^2 \cdot y = x \quad \text{é resolvida para um problema de valor de contorno } y(1)=3,$$

$y(11)=2.5$, $1 \leq x \leq 11$ e uma discretização com passo $h=2$ utilizando diferenças finitas

centrais para aproximar a derivada $\left(\frac{dy}{dx} \Big|_i \simeq \frac{y_{i+1}-y_{i-1}}{2(h)} \text{ e } \frac{d^2y}{dx^2} \Big|_i \simeq \frac{y_{i+1}-2y_i+y_{i-1}}{h^2} \right)$.

OBS.: Demonstrar todos os passos até a obtenção das equações.

$$\frac{y_{i+1}-2y_i+y_{i-1}}{h^2} + x \cdot \frac{y_{i+1}-y_{i-1}}{2h} - x^2 \cdot y_i = x, \quad x_i = \{1, 3, 5, 7, 9, 11\}$$

$$\frac{y_{i+1}-2y_i+y_{i-1}}{2^2} + x \cdot \frac{y_{i+1}-y_{i-1}}{4} - x^2 \cdot y_i = x \Rightarrow (1-x)y_{i-1} - (2+x^2)y_i + (1+x)y_{i+1} = 4x_i$$

$$y_0 = y(1) = 3, \quad y_5 = y(11) = 2.5$$

$$-2y_0 - 11y_1 + 4y_2 = 12, \quad i=1, x_1=3$$

$$-4y_1 - 27y_2 + 6y_3 = 20, \quad i=2, x_2=5$$

$$-6y_2 - 51y_3 + 8y_4 = 28, \quad i=3, x_3=7$$

$$-8y_3 - 83y_4 + 10y_5 = 36, \quad i=4, x_4=9$$

$$-11y_1 + 4y_2 = 12 + 2y_0$$

$$-4y_1 - 27y_2 + 6y_3 = 20$$

$$-6y_2 - 51y_3 + 8y_4 = 28$$

$$-8y_3 - 83y_4 = 36 - 10y_5$$

$$-11y_1 + 4y_2 = 18$$

$$-4y_1 - 27y_2 + 6y_3 = 20$$

$$-6y_2 - 51y_3 + 8y_4 = 28$$

$$-8y_3 - 83y_4 = 11$$

Questão 17

Considere a matriz $n \times n$ abaixo (também conhecida como matriz de Hilbert), onde

$$a_{ij} = \frac{1}{i+j-1}, \quad 1 \leq i, j \leq n :$$

$$\begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \dots & \frac{1}{n} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \dots & \frac{1}{n+1} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \dots & \frac{1}{n+2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \frac{1}{n} & \frac{1}{n+1} & \frac{1}{n+2} & \dots & \frac{1}{2n-1} \end{bmatrix}$$

Considere também a seguinte implementação do Método de Gauss-Seidel para solução de Sistemas Lineares:

```
1. // Método de Gauss-Seidel para solução de um sistema linear de ordem n
2. // A: matriz de coeficientes
3. // b: termos independentes
4. // x: solução do SL (pré-allocado com valores iniciais em 0)
5. // n: ordem do SL
6. // maxit: quantidade máxima de iterações
7. void gaussSeidel(double **A, double *b, double *x, int n, int maxit)
8. {
9.     double s;
10.    for (int k = 0; k < maxit; ++k) {
11.        for (int i = 0; i < n; ++i) {
12.            s = 0.0;
13.            for (int j = 0; j < i; ++j) s += A[i][j] * x[j];
14.            for (int j = i+1; j < n; ++j) s += A[i][j] * x[j];
15.            x[i] = (b[i] - s) / A[i][i];
16.        }
17.    }
18. }
```

- (a) Defina uma implementação alternativa deste método de Gauss-Seidel, específica para sistemas lineares com matrizes de coeficientes de Hilbert, de forma que a implementação não precise receber ou criar a matriz de coeficientes.
- (b) Em sua implementação, qual o acréscimo (ou decréscimo) na quantidade de operações em ponto flutuante (FLOPS) por iteração do método em relação à implementação original? Justifique sua resposta.

OPÇÃO 1

```
// b: vetor de termos independentes
// x: solução do SL (pré-alocado com valores iniciais em 0)
// n: ordem do SL
// maxit: quantidade máxima de iterações
void gaussSeidel (double *b, double *x, int n, int maxit)
{
    double s;

    for (int k=0; k < maxit; ++k) {
        for (int i=0; i < n; ++i) {
            s = 0.0;
            // aij = 1/( (i+1) + (j+1) - 1)
            for (int j=0; j < i; ++j) s += (1.0 / (i+j+1)) * X[j];
            for (int j=i+1; j < n; ++j) s += (1.0 / (i+j+1)) * X[j];
            x[i] = (B[i] - s) / (1.0 / (2*i+1)); // aii = 1/(2*(i+1)-1)
        }
    }
}
```

Quantidade FLOPS:

* $1/(i+j+1)$ acrescenta $n*(n-1)$ FLOP

* $1/(2*i+1)$ acrescenta n FLOP

* Resultado: Acréscimo de n^2 FLOP

OPÇÃO 2

```
// b: vetor de termos independentes
// x: solução do SL (pré-alocado com valores iniciais em 0)
// n: ordem do SL
// maxit: quantidade máxima de iterações
void gaussSeidel (double *b, double *x, int n, int maxit)
{
    double s;

    for (int k=0; k < maxit; ++k) {
        for (int i=0; i < n; ++i) {
            s = 0.0;
            // aij = 1/( (i+1) + (j+1) - 1)
            for (s=0, j=0; j < i; ++j) s += X[j] / (i+j+1);
            for (j=i+1; j < n; ++j) s += X[j] / (i+j+1);
            x[i] = (B[i] - s) * (2*i+1); // aii = 1/(2*(i+1)-1)
        }
    }
}
```

Quantidade FLOPS:

* Neste caso, não há acréscimo de FLOPS, pois ocorre apenas troca de operações:

- multiplicação para divisão ao calcular **s** nos laços internos
- divisão para multiplicação ao calcular **x[i]**

Questão 18

Considere a implementação do método da Fatoração LU abaixo, dividido em duas funções:

- i. **fatoracaoLU** fatora a matriz **A** do SL em 2 matrizes **L** e **U** (devolvida em **A**).
- ii. **resolveSL** recebe como parâmetros a matriz decomposta e um vetor **B** de termos independentes, e retorna em **X** a solução do SL **LUX=B**.

Proponha uma versão otimizada desse programa de forma a reduzir o uso de memória a uma única matriz **nxn** ao invés de duas. Faça as adaptações necessárias para que o método funcione corretamente. Reescreva o código com as otimizações propostas.

```
1. void fatoracaoLU(double **A, double **L, int n)
2. {
3.     zera_matriz(L, n);
4.
5.     for (int i = 0; i < n; ++i)
6.         for (int k = i+1; k < n; ++k) {
7.             double m = A[k][i] / A[i][i];
8.             L[k][i] = m;
9.             for (int j = i; j < n; ++j)
10.                A[k][j] -= A[i][j] * m;
11.        }
12.    for (int i = 0; i < n; ++i)
13.        L[i][i] = 1;
14.}
15.
16. void resolveSL(double **L, double **U, double *B, double *X, int n)
17. {
18.     double *Y = aloca_vetor(n);
19.
20.     // resolve o sistema LY = B
21.     for (int i = 0; i < n; ++i) {
22.         Y[i] = B[i];
23.         for (int j = 0; j < i; ++j)
24.             Y[i] -= L[i][j] * Y[j];
25.         Y[i] /= L[i][i];
26.     }
27.     // resolve o sistema UX = Y
28.     for (int i = n-1; i >= 0; --i) {
29.         X[i] = Y[i];
30.         for (int j = i+1; j < n; ++j)
31.             X[i] -= U[i][j] * X[j];
32.         X[i] /= U[i][i];
33.     }
34.     free(Y);
35. }
```

SOLUÇÃO: armazenar **L** e **U** na mesma matriz.

```
void fatoracaoLU(double **A, int n)
1. {
2.     for (int i = 0; i < n; ++i)
3.         for (int k = i+1; k < n; ++k) {
4.             double m = A[k][i] / A[i][i];
5.             A[k][i] = m;
6.             for (int j = i+1; j < n; ++j)
7.                 A[k][j] -= A[i][j] * m;
8.         }
9. }

10.
11. void resolveSL(double **LU, double *B, double *X, int n)
12. {
13.     double *Y = aloca_vetor(n);
14.
15.     // resolve o sistema LY = B
16.     for (int i = 0; i < n; ++i) {
17.         Y[i] = B[i];
18.         for (int j = 0; j < i; ++j)
19.             Y[i] -= LU[i][j]*Y[j];
20.     }
21.
22.     // resolve o sistema UX = Y
23.     for (int i = n-1; i >= 0; --i) {
24.         X[i] = Y[i];
25.         for (int j = i+1; j < n; ++j)
26.             X[i] -= LU[i][j] * X[j];
27.         X[i] /= LU[i][i];
28.     }
29.     free(Y);
30. }
```